Data Contract — the interface between the two tracks

Version: 1.0 (Visium HD geometry, delivered at 2 µm squares)

This document defines the *contract* that connects the two project tracks. The **Simulation track** produces data in this format; the **Analysis track** consumes data in this format. Because both sides agree on the contract, a simulator built by one student can be analyzed by a tool built by another — and, crucially, the simulator ships the **ground truth** that lets the analyzer check whether it is right.

> **Why a data contract at all?** A well-organized computational project separates *what the data looks like* from *what the code does with it*. If the two tracks negotiated formats informally, every change would break the other side. By fixing an explicit, versioned contract we get exactly the property research software needs: components that can be developed, tested and replaced independently. This is the same discipline behind file formats like FASTQ, BED, or AnnData.

A note on formats. Everything here is plain CSV and JSON. Real tools use HDF5-backed containers such as AnnData (`.h5ad`), which store sparse matrices efficiently and keep annotations aligned automatically. We use CSV deliberately: you can open it in any editor and see exactly what your program wrote, which matters far more here than efficiency.

---

1. Scope

This document specifies **file formats only**. The science behind them — what spatial transcriptomics measures, what a bin is, why a single-cell reference is needed, what deconvolution and the scoring metrics mean — is in the **background primer**, which is on ADAM. Read that first; this document assumes it and does not repeat it.

The data model in one line: the tissue is measured as a grid of **2 µm squares**, each holding the counts of the transcripts captured above it, and the simulator additionally records which cell type each of those transcripts came from.

The dataset is delivered at the instrument's resolution, not at an analysis resolution. Summing squares into larger **bins** (8 µm, 16 µm, or anything else) is a decision each program makes for itself and records in its own metadata. That is why nothing here fixes a bin size: bin size is an analysis parameter, not a property of the experiment.

---

2. Directory layout produced by the simulator

```
dataset_<name>/
├── metadata.json            # format version, square size, provenance (REQUIRED)
├── coordinates.csv             # position of each square on the grid (REQUIRED)
├── counts.csv                  # squares × genes count matrix (REQUIRED)
├── reference/               # LEGITIMATE INPUT — stands in for reference scRNA-seq
│   ├── reference_counts.csv    # counts for individually profiled cells
│   └── reference_labels.csv    # the cell type of each reference cell
└── ground_truth/             # the "answer key" — the analyzer must NOT read this
    ├── params.json             # exact simulation parameters + random seed
    ├── composition.csv        # true transcript counts per cell type, per square
    └── domains.csv             # true spatial-domain label per square
```

Golden rule for the Analysis track: your tool receives everything *except* `ground_truth/`. You may open `ground_truth/` only in **scoring and test** code, never in inference code. Keeping

the two separated is what makes your correctness checks honest — the same reason raw data is kept read-only and separate from results.

`reference/` stands in for the separate single-cell experiment the primer describes. It is a legitimate input: the analyzer estimates its signatures from it, and must **not** take them from `ground_truth/params.json`, which is scoring-and-test material only (§3.5).

Ground truth is deliberately aggregate-only. The simulator knows each cell's exact position and which cell generated each transcript while it runs, but none of that survives to `ground_truth/` — only the per-square, per-cell-type transcript *count* (§3.6). This is sufficient for the two metrics this course scores (RMSE/JSD on composition, ARI on domains) and keeps the ground truth small and unambiguous. It does mean `ground_truth/` cannot be used to audit the aggregation step itself independently of the code that produced it, or to score anything at finer-than-square resolution — a real limitation, not an oversight.

---

3. File specifications

3.1 `metadata.json` (REQUIRED)

```
{
  "contract_version": "1.0",
  "square_size_um": 2,
  "grid": {"n_rows": 160, "n_cols": 160},
  "field_um": {"width": 320.0, "height": 320.0},
  "n_squares": 25600,
  "n_genes": 100,
  "cell_types": ["A", "B", "C"],
  "created_utc": "<ISO-8601 UTC timestamp>",
  "generator": {"name": "tissue-sim", "version": "<your package version>",
                "git_commit": "<output of: git rev-parse --short HEAD>"},
  "random_seed": 20260725
}
```

Values in angle brackets are placeholders your program fills in at run time; the others are illustrative.

`contract_version` MUST be checked by the analyzer on load. `square_size_um`, `git_commit` and `random_seed` are what make a dataset reproducible and interpretable — record them or you cannot regenerate your own data. Note that `git_commit` is only meaningful if the working tree was clean when the dataset was generated; recording it from a dirty tree records a lie, so it is worth checking.

3.2 `coordinates.csv` (REQUIRED)

One row per square. Squares tile the field on a regular grid with no gaps.

column	type	description
`square_id`	str	primary key, e.g. `sq_r007_c010`
`array_row`	int	row index on the grid, 0-based
`array_col`	int	column index on the grid, 0-based
`x_um`	float	x coordinate of the square center, in µm
`y_um`	float	y coordinate of the square center, in µm

Row and column indices are the primary spatial reference; the µm coordinates are derived (`x_um = (array_col + 0.5) * square_size_um`) and are there so that distances are meaningful. A bin of side `k * square_size_um` is the block of `k × k` squares with `array_row // k` and `array_col // k` equal — which is all a program needs in order to bin.

3.3 `counts.csv` (REQUIRED)

Integer count matrix. First column `square_id`; remaining columns are gene IDs. Values are non-negative integers. Row order matches `coordinates.csv`.

```
square_id,GENE_0001,GENE_0002,...,GENE_0100
sq_r000_c000,0,1,...,0
sq_r000_c001,0,0,...,2
```

A 2 µm square captures very few molecules, so this table is mostly zeros — which is true of the real data too, and is the reason production formats store it sparsely. Binning is what makes the numbers usable.

3.4 `reference/reference_counts.csv` and `reference/reference_labels.csv` (REQUIRED)

A simulated single-cell experiment: individually measured whole cells with known types.

`reference_counts.csv` — same layout as `counts.csv`, one row per reference cell (`ref_0001`, ...), the same gene columns in the same order.

`reference_labels.csv`

column	type	description
`cell_id`	str	reference cell id
`cell_type`	str	its cell type

Requirements: at least ~50 cells per type, so signatures can be estimated with reasonable error; drawn **independently** of the cells used to build the tissue; covering every cell type present in the tissue. The number of reference cells per type is a parameter — shrinking it is a legitimate way to make the problem harder.

3.5 `ground_truth/params.json`

Written by the **Simulation track** as part of generating a dataset — it falls out for free once the simulator has run. Two things read it afterward: any benchmark table, which §5.2 requires to report the difficulty settings recorded here; and anyone who wants to regenerate this exact dataset later, since the full parameter set plus the seed is enough to do that. No inference code reads it, ever (§2).

At minimum, present with these exact keys:

key	type	description
`seed`	int	matches `metadata.json`'s `random_seed`
`cell_types`	list	strings, same order as `metadata.json`
`expression_programs`	list	one list of `n_genes` floats per cell type
`marker_genes`	object	cell type → list of that type's marker gene indices
`n_markers_per_type`	int	marker genes given to each type (simulation spec F2)

key	type	description
`marker_gene_boost`	float	fold-change applied to a marker over its own background (simulation spec F2)
`reference_n_per_type`	int	reference cells simulated per type

`marker_genes` is here because the marker sets may be **drawn** rather than listed in the config: which genes mark which type is then a result of the run, and a test that asks whether a type's markers really are elevated in that type has no way to know which genes to look at otherwise. Gene indices are 0-based and index into `counts.csv`'s gene columns in order.

`expression_programs` is the true, exact expression distribution for each cell type — the probability vector `p_t` that every cell of that type draws its transcripts from, in `cell_types` order, each list summing to 1. It is fixed once, when the simulator builds the cell types, before any transcripts are sampled — it is a *parameter* of the model, not something computed by looking back at what got sampled. Each cell then draws its own, noisy, finite number of transcripts from `p_t`, and that per-cell sampling noise is real and expected — it's why the Analysis track cannot just read off the answer even from a large reference — but it is not itself part of the ground truth; only the clean distribution that generated it is.

A config file does not usually spell out `expression_programs` gene-by-gene — it exposes scalar knobs such as `n_markers_per_type`, `marker_gene_boost` and the spread of the baseline abundance distribution, and the simulator turns those into concrete per-gene numbers using the seed. `expression_programs` records the result of that process. That's the sense in which it isn't just "the config": you cannot get these numbers by reading the config file, only by running the simulator (or reading them back out of `params.json`).

A simulator MAY record additional parameters beyond these seven — field size, cell density, cell radius, domain layout, and anything else the simulation spec's F2 asks it to expose — since this file's job is to be read by scoring and test code, not to be a fixed interchange format. The seven keys above are the ones this contract and the analysis track's tests depend on by name; anything else is free-form.

Bin size is never one of these fields. It is an analysis-time choice (§1), never a property of the simulated dataset, so it has no place in `params.json` at all.

3.6 `ground_truth/composition.csv`

Long format, one row per (square, cell_type) with a non-zero count.

column	type	description
`square_id`	str	square id
`cell_type`	str	a cell type contributing transcripts to this square
`n_transcripts`	int	how many of the square's transcripts came from that cell type

Within a square these MUST sum to the square's total in `counts.csv`.

Absence means zero. A `(square_id, cell_type)` pair not listed here contributed zero transcripts to that square — the file is sparse by design, not incomplete. Any consumer (scoring code, `validate_dataset`, an aggregation step) MUST treat a missing pair as zero, not skip it and not treat its absence as a validation failure. A square with zero total transcripts legitimately has zero rows for it in this file at all, and that is consistent, not a missing-data bug — its `counts.csv` total is zero too.

Counts, not fractions, and transcripts, not cells. Counts are stored because they aggregate: to obtain the truth at any bin size, sum them over the block and divide at the end. Fractions cannot be combined that way — averaging them is wrong whenever the squares hold different totals. (Two squares, one with 100 transcripts of type A and one with a single transcript of type B: the fractions average to 0.5 / 0.5, while the correct answer for the pair is 0.99 / 0.01.) Why the quantity is transcripts rather than cells is explained in the primer, where it works through deconvolution.

3.7 `ground_truth/domains.csv`

column	type	description
`square_id`	str	square id
`domain`	str	true spatial-domain label

---

4. What the Analysis track outputs

```
results_<name>/
├── predicted_composition.csv   # bin_id, cell_type, fraction
├── predicted_domains.csv       # bin_id, domain
└── run_metadata.json           # contract version consumed, tool version,
                                # parameters, and the bin_size_um used
```

Predictions are reported at the **bin** size the analysis chose, not at 2 µm. A bin's id is `bin_r<row>_c<col>` with the row and column of the bin grid, so that the mapping back to squares is unambiguous given `bin_size_um`, which the analyzer MUST record in `run_metadata.json`. Predicted fractions are per bin and sum to 1 (± 1e-3).

---

5. Scoring

The metrics both tracks agree to report. The primer defines all three, in the section on the questions this data answers.

Task	Metric	Perfect	Chance level
Deconvolution	mean per-bin **RMSE** of the fractions	0.0	~0.3–0.5
Deconvolution	mean per-bin **Jensen–Shannon divergence**	0.0	high
Spatial domains	**Adjusted Rand Index (ARI)** vs true labels	1.0	~0.0

Because predictions are made at a bin size of the analysis's choosing while the truth is stored per square, the two must be brought to a common footing. The rule differs by quantity, and in each case it is exact:

- **Composition — bring the truth up.** Sum the ground-truth transcript counts over the squares in each bin, then convert to fractions. Summing counts is exact; averaging fractions is not (§3.6). Bins with no transcripts are excluded.
- **Domains — bring the prediction down.** A bin's predicted label applies to every square inside it, so ARI is computed per square against `domains.csv`. This needs no rule for bins that straddle a boundary, and it means a coarse binning is charged for the boundary detail it blurs — which is a real cost and should appear in the score.

A consequence worth noting: scores computed at different bin sizes are directly comparable, because both are measured against the same 2 µm truth.

5.1 The aggregation invariant

Because binning is exact summation, as the primer explains, any program that bins the data must satisfy, gene by gene and with no tolerance:

```
counts_16[r, c] == counts_8[2r,   2c] + counts_8[2r,   2c+1]
                 + counts_8[2r+1, 2c] + counts_8[2r+1, 2c+1]
```

and likewise for the ground-truth transcript counts. Since both tracks bin the same data, each implements this independently — so the invariant also serves as a differential check between the two tracks: two correct implementations must produce identical bins.

5.2 Reported difficulty

A score is meaningless without the settings that produced it. Any benchmark table MUST carry, for each row, the values of the parameters that were swept, taken from `ground_truth/params.json` (§3.5) — at minimum the three named `marker_gene_boost`, `n_markers_per_type` and `reference_n_per_type`. Bin size is not one of them: it is an analysis-side setting rather than a property of the dataset, so it never appears in `params.json`, but a benchmark table still varies it and each row records the value used. (What these do to the difficulty is the primer's section on what makes the problem hard; which of them the simulator must expose is in the simulation spec.)

---

6. Versioning and change control

- The contract is **versioned** (`contract_version`). Breaking changes bump the major version. Both tracks must agree before changing it.
- Proposed changes go through a pull request against this file, reviewed by at least one student from each track. (This is the Week 12 collaboration workflow in action.)

---

7. Minimal conformance checklist

A dataset conforms to the contract if:

- ☐ `metadata.json` is present and valid JSON, with `contract_version` == "1.0"
- ☐ `coordinates.csv`, `counts.csv` are present with the required columns
- ☐ `array_row` / `array_col` cover the full grid without gaps or duplicates
- ☐ `x_um`, `y_um` agree with the indices and `square_size_um`
- ☐ `square_ids` are consistent across all files, in the same order
- ☐ every count is a non-negative integer
- ☐ `reference/` is present, labeled, and covers every cell type in the tissue
- ☐ `ground_truth/` is present, and each square's per-type transcript counts sum to that square's total in `counts.csv`

Both tracks implement this checklist as a single function, `validate_dataset(path)`: the Simulation track calls it on the directory it has just written, as its output guarantee, and the Analysis track calls it on the directory it is about to read, as its first correctness check. Same contract, same checks, one name.